Aura Frontend Midterm Reviewer

Interview guide for a student assigned to Frontend and Deployment responsibilities.

Vue 3 | Vite | Vue Router | API Integration | Docker | Deployment

1. Role Summary

You can introduce yourself like this:

> I am responsible for frontend integration and deployment readiness. I worked on connecting the Vue frontend to the real backend, fixing broken user flows, organizing role-based navigation, and preparing the project so it can run in Docker and be demonstrated consistently.

Role Area	What I Did	Purpose	Why It Matters
Attendance	Integrated student sign-in, waiting-for-sign-out, sign-out, and final attendance state with backend rules.	Make attendance accurate and match real event phases.	Wrong state here affects attendance records, analytics, and email notifications.
SG/SSG Dashboard	Separated SG workspace routing from normal student dashboard routing.	Keep role-based views organized and prevent broken navbar behavior.	Users need the correct dashboard and navigation context based on role.
Campus Admin	Connected student import, event management, reports, and program student management to the backend.	Allow real admin workflows instead of mock or broken actions.	This is the main operational side used by school administrators.
Public Attendance	Added Quick Attendance from the login side without requiring a user session.	Support public face attendance for nearby events.	Important for kiosk-like attendance use cases.
Deployment	Prepared Docker, nginx proxying, runtime config, and environment-backed startup.	Make the project easier for other teammates to run and demo.	Deployment readiness is part of software ownership, not just coding.

2. Frontend Concepts You Must Explain

Term	Meaning	Frontend Example	Purpose
API	An interface the frontend uses to talk to the backend and exchange data.	Frontend requests events, users, attendance records, and imports.	Connect UI to real system data.
Endpoint	A specific backend URL for one action or one resource.	/api/events/, /api/attendance/me/records	Defines exactly where a request goes.
Request Payload	The data sent from frontend to backend.	Creating an event with name, time, geofence, and scope data.	Tells the backend what to create or update.
Response	The data returned by the backend after a request.	Attendance status, imported row summary, event list.	Lets the UI update based on real backend state.
CORS	Cross-Origin Resource Sharing. A browser security rule controlling whether a frontend can call a backend from another origin.	Frontend on one host calling backend on another host.	Without correct CORS or proxying, requests fail in the browser.
Proxy	A middle layer that forwards frontend requests to the backend.	The app uses /__backend__ and nginx or Vite proxying.	Avoids CORS issues and keeps config flexible.

3. How Frontend Connects to the API

> The frontend does not talk directly to the database. It sends HTTP requests to backend endpoints. In Aura, the frontend usually calls the backend through /__backend__ using runtime configuration, then service files normalize the responses so Vue components can display clean data.

Step	What Happens	Aura Example	Why This Is Good Practice
1. UI action	User clicks a button, submits a form, or opens a page.	Student clicks Attendance Now.	Business flow starts from a visible user event.
2. Service call	A frontend service file sends the request.	backendApi.js or publicAttendance.js	Keeps API logic separate from UI.
3. Backend processing	The backend validates, computes rules, and returns a result.	Attendance phase decides sign-in, waiting, or sign-out.	Important logic stays server-side, not hardcoded in UI.

Step	What Happens	Aura Example	Why This Is Good Practice
4. Normalize result	The frontend cleans response shape if needed.	Attendance and import responses are normalized before use.	Makes components simpler and more stable.
5. Update UI	Vue state changes and the screen reacts.	Button changes to Waiting for Sign Out.	Gives users correct feedback immediately.

4. Important API and Frontend Terms

What is an Endpoint?

An endpoint is a specific backend address used for one task.

- GET /api/events/ gets events
- PATCH /api/events/{id} updates an event
- DELETE /api/events/{id} deletes an event

What is CORS?

CORS is a browser security rule. If the frontend and backend are on different origins, the backend must allow the frontend origin, or the browser blocks the request.

How Aura avoids CORS problems

- Vite development uses a proxy through /__backend__.
- Docker nginx also proxies /__backend__ to the backend host.
- This makes the browser see the requests as coming from the same frontend origin.

What happens without correct API integration?

- Buttons work visually but data is not saved.
- Wrong role gets the wrong screen.
- Attendance status becomes misleading.
- Import preview works but actual import fails.

5. Project-Specific Work You Can Defend

Feature	Your Contribution	Backend/Frontend Connection	Possible Interview Question
Attendance flow	Fixed sign-in, waiting-for-sign-out, sign-out, and final dashboard count behavior.	Frontend now follows backend attendance phases instead of guessing from event status.	Why should attended only count after sign-out?
SG/SSG navigation	Fixed navbar and route context so SG workspace does not break student dashboard.	Used role-aware routing and workspace context in Vue Router.	What was causing the white screen or wrong dashboard?
Student import	Matched the preview-token import flow and improved success or error handling.	Frontend previews file first, then imports using the backend preview token.	Why not upload directly to import immediately?
Event management	Made edit and delete functional for campus admin and SG with a shared event editor UI.	Uses real PATCH and DELETE event endpoints.	What fields can actually be edited from the frontend?
Public attendance	Added Quick Attendance to login side without requiring a session.	Connects to public attendance endpoints for nearby events and multi-face scanning.	How can attendance happen without user login?

6. Common Interview Questions with Suggested Answers

Question	Suggested Answer
What is your role in the team?	I am responsible for frontend integration and deployment readiness. I connect the UI to the backend, fix broken user flows, and make the app stable enough to run in browser and Docker demo environments.
What did you personally do?	I worked on attendance integration, SG or SSG routing, campus admin event management, student import, Quick Attendance, and Docker-based deployment setup.
Why did you avoid hardcoding?	Because attendance phases, role scope, import validation, and event rules belong to backend logic. Hardcoding would make the frontend fragile and inaccurate when backend data changes.
What will break if your work is removed?	Attendance state would become inconsistent, SG workspace could route incorrectly, admin event actions may stop working, import could fail, and deployment would become harder to reproduce.
What was your biggest challenge?	The hardest part was aligning the frontend to the real backend contracts. Many bugs were caused by small mismatches in endpoint path, payload shape, or expected response state.

Question	Suggested Answer
What improvement would you suggest next?	I would add automated end-to-end tests for attendance, import, and SG routing because these are multi-step flows with high regression risk.

7. Demo Checklist

- Show login and role-based access.
- Show student schedule and attendance button state changes.
- Show SG dashboard and event management flow.
- Show campus admin import preview and actual import.
- Show event reports and attendance records.
- Show Quick Attendance from login side.
- Explain Docker startup and backend proxying.

8. Docker and Deployment Reviewer

File	Purpose	How to Explain It
docker-compose.yml	Starts the frontend container with backend-related environment variables.	This is the main deployment entry for local demo or team testing.
Dockerfile	Builds the Vue app with Node, then serves it with nginx.	This creates a production-like frontend container instead of running Vite dev mode.
nginx.conf.template	Handles SPA routing and proxies /__backend__ to the backend host.	This is how we support route refresh and avoid frontend or backend origin issues.
.env.docker	Stores backend origin and port values used by Docker.	Teammates can change deployment target without editing source code.

Command to start: docker compose --env-file .env.docker up --build -d

9. Final 1-Minute Summary

> My main contribution is frontend integration and deployment readiness. I connected the Vue frontend to the real backend, fixed role-based navigation, attendance logic, event management, import flows, and public attendance. I also helped make the project easier to run through Docker, nginx proxying, and environment-based configuration. My approach was to follow backend contracts instead of hardcoding frontend assumptions.

Prepared for Aura Midterm Interview Review. This document focuses on Frontend and Deployment responsibilities, using project-specific examples and simple explanations for live oral defense.